Natural-Language Routing on an Open Stack

John Gunnar Carlsson Andreas Royset

FOSS4G 2026
Wednesday, November 4

Maps expose toggles, but requests are more fluid

- What navigation products accept:
  - global switches: avoid highways, tolls, ferries
  - search ranked by distance from the user ("coffee near me")
- What people actually ask for:
  - "gas on the right, so I don't turn across traffic"
  - "coffee about an hour from now"
  - "stay off 880"
  - "between the two of us, who picks up our kid?"
- Each is a constraint expressible in plain language and computable with open tools

Operations research in one slide

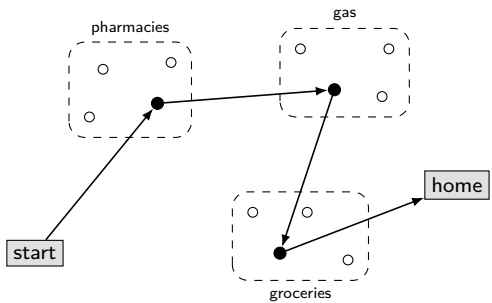
- A model: decision variables, constraints, an objective

$$\min_{x \in X} f(x) \text{ subject to } g_i(x) \leq 0$$

- The modelling is the hard part; solvers are interchangeable behind it
- Families: linear programs (LP), mixed-integer programs (MIP), constraint programming (CP), local search
- Classic uses: airline crews, supply chains, scheduling, network design, and vehicle routing
- Most routing problems are NP-hard; practical instances are solved by good heuristics with a time limit, not by proofs of optimality

Vehicle routing: a family of problems

- TSP: one tour through all nodes, minimum length
- VRP: several vehicles share the nodes
- VRPTW: each node has an arrival window $[a_i, b_i]$
- Pickup and delivery: pairs (p, d), same vehicle, p before d
- Generalized TSP: nodes come in clusters; visit exactly one per cluster
- Prize-collecting: nodes are optional; skipping one costs a penalty
- All of these are one solver configured differently



visit exactly one node per cluster;
the solver picks the member and the order together

Google OR-Tools

- Open source (Apache 2.0), maintained by Google
- CP-SAT: constraint programming over a SAT core; many MiniZinc Challenge gold medals
- Linear and mixed-integer: one interface over GLOP, PDLP, SCIP, HiGHS, and commercial solvers
- Graph algorithms: shortest paths, max flow, min-cost flow, assignment
- Routing: a vehicle-routing library on top of a CP solver, with local search
- We use the routing library; the same install would also schedule, pack and assign

The routing library, in OR terms

Concept	In the model	OR-Tools name
Arc cost	t_{ij}, driving times from Valhalla	transit callback
Time along a route	$T_j \geq T_i + s_i + t_{ij}$ on arcs used	time dimension
Time window	$a_i \leq T_i \leq b_i$	bounds on T_i
One of a cluster	$\sum_{i \in G_k} y_i = 1$	disjunction
Optional node	skip i at penalty π_i	disjunction, penalty π_i
Pickup before drop	same vehicle, $T_p \leq T_d$	pickup–delivery pair
Makespan	$\max_v T_v^{\text{end}} - \min_v T_v^{\text{start}}$	span cost

- Search: a construction heuristic, then a metaheuristic (guided local search, simulated annealing, tabu) until a time limit
- No lower bound: the answer is a good solution, not a certified optimum

The five demos as optimization problems

Request	Problem	Solved by
1. Gas on the right	best of n filtered candidates	one matrix, minimum
2. Coffee an hour in	same, filtered by time on route	one matrix, minimum
3. Stay off 880	shortest path, edges removed	re-solve until no avoided edge
4. Pharmacy, groceries, gas	generalized TSP, time windows	OR-Tools, 1 vehicle
5. Split with my wife	pickup–delivery, min total + span	OR-Tools, 2 vehicles

- OR-Tools only where order and choice interact; the rest is exact and instant
- Stops in a stated order: layered shortest path, exact in $O(km^2)$ for k stops of m candidates
- Demo 4 has 1296 tours; local search finds a good one in 1 s with no proof, and the verifier checks the deadlines, not optimality

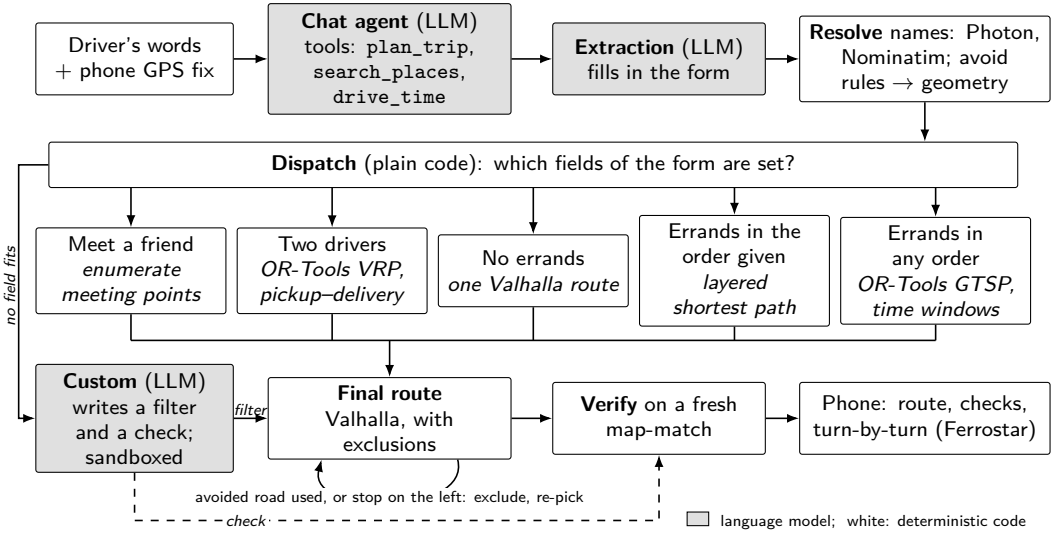
Approach: the model fills in a form

- A language model reads the request and fills in a fixed form
- The fields and their allowed values are set in advance; the model cannot answer outside them
- Ordinary code turns the form into an optimization model and solves it
- The model never routes or optimizes; it sees no coordinates and no store names
- The start is the phone's GPS position
- Every result is re-checked against the final route

"I need a pharmacy before it closes at six, groceries, and gas, in any order, and I have to be home by 6:15."

Field	Value
start	the phone's position
end	home
errand	pharmacy, closes 18:00
errand	groceries
errand	gas
order	any
arrive by	18:15

How one request runs



One request, end to end

"I need a pharmacy before it closes at six, groceries, and gas, in any order, and I have to be home by 6:15." (5:15 pm, Great Mall)

	Stage	Done by	Output	Time
1	Read the request	language model	it is a trip: plan it	
2	Fill in the form	language model	3 errands, 2 deadlines	2.8 s
3	Choose the method	fixed rules	errands, any order $\Rightarrow$ GTSP	
4	Places	saved places	home = Milpitas Library	
5	Direct route	Valhalla	7.2 min	
6	Candidates	Overture places	7 pharmacies, 27 groceries, 15 gas	
7	Screen	detour of each	best 6 per errand	
8	Travel times	Valhalla	20×20 matrix	0.34 s
9	Solve	OR-Tools	Nilgiris, Walgreens, 76; 10.4 min	1.0 s
10	Check	Valhalla map-match	5 of 5 checks pass	
			optimization total 1.4 s; request total	6.9 s

Steps 1–2: the two model calls

- The agent chooses a tool: place search, or drive time
- Its prompt sends errands and task splits to `plan_trip` unmodified, so the optimizer chooses the stores
- Extraction fills a fixed JSON schema, enforced by the API (`response_format: json_schema`; Ollama: `format`)
- Then a sanitizer: enums checked, numbers clamped, missing fields defaulted
- No coordinates or store names in the form, only words and times

```
// 1. agent -> tool call
{"tool": "plan_trip", "args": {"request":
  "pharmacy, groceries, and gas, in any
   order, pharmacy must be before 6 PM,
   home by 6:15 PM"}}

// 2. extraction -> the form (trimmed)
{"origin": null,  "destination": "home",
 "stopOrder": "any", "arriveBy": "18:15",
 "stops": [
   {"kind": "errand", "what": "pharmacy",
    "closesAt": "18:00", "brands": []},
   {"kind": "errand", "what": "groceries"},
   {"kind": "errand", "what": "gas"}],
 "team": null, "meetup": null,
 "custom": null}
```

Step 3: the form determines the problem

- The model never names an algorithm; it answers narrow questions about the text ("did they give an order?")
- Fixed rules read the form, top to bottom
- Same form, same method: a wrong method traces back to one wrong field
- Here: errands, any order $\Rightarrow$ GTSP with time windows

The form has	Problem	Solved by
a friend to meet	choose a meeting point	enumeration
tasks for two drivers	VRP, pickup–delivery	OR-Tools
a rule with no field	model-written filter	then as below
no errands	shortest path	Valhalla
errands, order given	layered shortest path	dynamic program
errands, any order	GTSP, time windows	OR-Tools

What each field becomes

Field of the form	In the optimization model
errand (a kind, or chains)	cluster G_k of candidates from Overture; visit exactly one
named place	one node, geocoded
closes at, arrive by	time window: $T_i \leq b_i$
about N minutes in	candidates near where the direct route is at time N
on the right	within 80 m, right of travel; re-pick on arrival
maximum detour D	candidates only with $\delta(p) \leq D$
avoid a road or area	edges removed; re-solve until no avoided edge is used
pick someone up	pickup–delivery pair: same vehicle, $T_p \leq T_d$
two drivers	second vehicle; objective adds the makespan
anything else	a model-written filter and check, run in isolation

New combinations need no new code; a new kind of constraint needs a field and its translation.

Steps 4–8: from words to a 20-node instance

- Candidates: Overture places within 2 km of the direct route
- Screen by detour; keep the best 6

$$\delta(p) = t(o, p) + t(p, d) - t(o, d)$$

- Instance: start, home, 3×6 candidates $= 20$ nodes
- One 20×20 travel-time matrix from Valhalla (0.34 s; typical speeds in this run)
- Choices: 6^3 stores $\times 3!$ orders $= 1296$ tours

Errand	Found	Kept	Best two, δ (min)
pharmacy	7	6	Walgreens 2.8, CVS 6.5
groceries	27	6	New India Bazar 0.8, Nilgiris 1.2
gas	15	6	76 1.6, City Gas 2.5

Screening is per errand; the solver then sees the interactions: the chosen grocery is not the one with the smallest detour on its own

Steps 9–10: solve, then check independently

- Model: 1 vehicle, start and end fixed, 3 clusters of 6, pharmacy by 6:00, home by 6:15
- Search: cheapest-arc construction, guided local search, 1 s limit
- Result: 10.4 min (direct: 7.2)
- Re-costed from the matrix; the solver's objective is not trusted
- Check on a fresh map-match of the final route: each stop within 20 m of it, both deadlines met

Stop	Arrive	Limit
Great Mall	5:15	
Nilgiris (groceries)	5:21	
Walgreens (pharmacy)	5:22	6:00
76 (gas)	5:23	
home	5:25	6:15

Tools

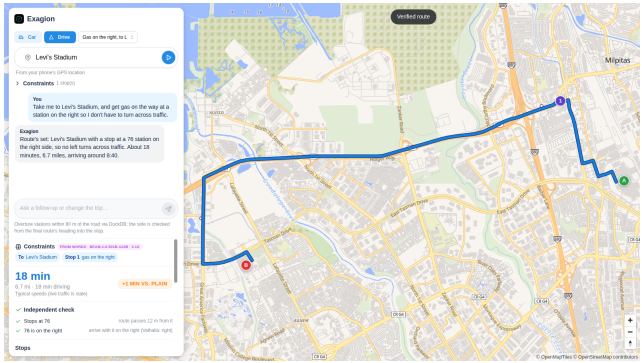
- Valhalla (OpenStreetMap road graph): routes, travel-time matrices, map-matching, road exclusions, time-dependent travel times
- Overture Maps places (273k in the Bay Area), queried with DuckDB
- Photon and Nominatim: OSM geocoding, self-hosted
- OR-Tools routing: disjunctions, time windows, pickup and delivery
- Language model: any hosted model with a standard API, or one run locally with Ollama
- Demos use Reflection AI's Beam-1.0 (501B-A23B), chosen because its weights are open; 3–14 s per request
- Caltrans PeMS loop detectors: hour-of-week speeds loaded as Valhalla predicted traffic
- Ferrostar: turn-by-turn navigation in the browser
- The local model works too: all five demos run on qwen3.6 35B-A3B through Ollama on one laptop with a 12 GB GPU (18–61 s each)

Five requests from the Great Mall, Milpitas

- Start: a GPS fix posted by the phone; no request states where the driver is
- Requests, and what each exercises:
  1. Gas on the right, on the way to Levi's Stadium
     DuckDB on Overture; arrival-side geometry
  2. Coffee about an hour into the drive to Sacramento
     maneuver times; Valhalla matrix
  3. Oakland airport, but stay off 880
     map-match, exclude, re-route
  4. Pharmacy before 6, groceries, gas, home by 6:15
     OR-Tools: disjunctions and time windows
  5. Split the afternoon's tasks between two drivers
     OR-Tools: two vehicles, pickup and delivery
- All screenshots come from the live path (model in the loop), via `scripts/capture_talk.py`

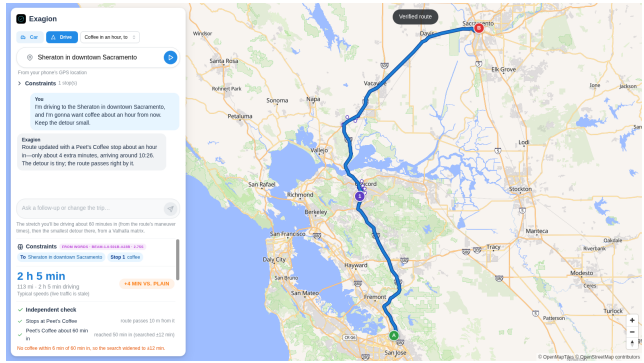
1. “Gas on the right, on the way”

- Candidates: Overture stations within 80 m of the road, right of travel
- Decision: heading of the final route over its last 15 m into the stop
- Arrival on the left ⇒ reject and re-pick
- Result: a 76 on W. Calaveras Blvd, +0.5 min
- To SJC: no candidate (freeway the whole way); reported as such



2. “Coffee about an hour from now”

- Model output: `afterMinutes: 60`
- Search the stretch driven 54–66 min in, from per-maneuver times
- Empty (Benicia bridge) ⇒ widen to ± 12 min, with a note
- Smallest detour: Peet’s, Pleasant Hill, $+0.3$ min, reached at 50 min
- Destination: the conference hotel



Candidates and detours

- Prefilter by distance to the route (2 km; 80 m for "on the right")
- Decide by time:

$$\delta(p) = t(o, p) + t(p, d) - t(o, d)$$

- Two thin matrices ($1 \times n$, $n \times 1$) screen n candidates; the best 6 per errand go on
- Time window: with $T(x)$ the elapsed time at x on the base route,

$$S(\tau) = \{x : |T(x) - \tau| \le w\}$$

with $w = \max(5, 0.1\tau)$ min, widened to $2w$, $3w$ if empty

```
SELECT id, name, category, lat, lon
FROM read_parquet('places.parquet')
WHERE lat BETWEEN ? AND ?
  AND lon BETWEEN ? AND ?
  AND category IN ('coffee_shop', 'cafe')
  AND coalesce(confidence, 0) >= 0.5

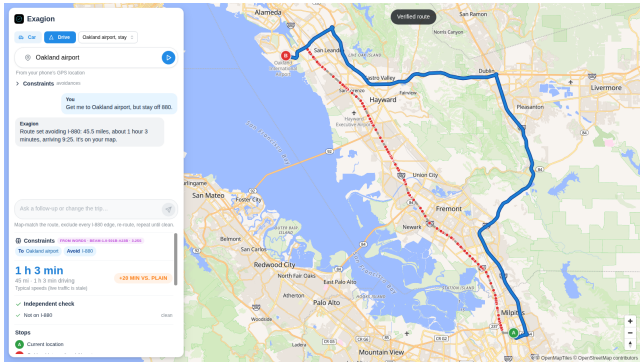
# shapely: offset and position on the route
offset = chunks[tree.nearest(pt)].distance(pt)
along  = line.project(pt) / line.length

# Valhalla /sources_to_targets
to_c   = matrix([origin], cands)[0][0]
from_c = [r[0] for r in matrix(cands, [dest])[0]]
detour = [a + b - direct
          for a, b in zip(to_c, from_c)]
```

`nlroute/places.py`, `planner.py` (trimmed)

3. “Oakland airport, but stay off 880”

- “Avoid I-880” is a predicate on graph edges, not a toggle
- Route → map-match → exclude violating edges → re-route
- Result: I-680 and I-580, 63 min; +20 min over the plain route (dotted)
- First run resolved “Oakland airport” to an airport hotel; a named kind must now match the OSM tag



Avoidance as a fixed-point iteration

- Valhalla excludes points, not predicates. With rule φ:

$$
\begin{aligned}
X_0 &= \emptyset \\
r_k &= \mathrm{route}(o, d;\, X_k) \\
B_k &= \{e \in E(r_k) : \varphi(e)\} \\
X_{k+1} &= X_k \cup m(B_k)
\end{aligned}
$$

  m: edge midpoints. Stop when $B_k = \emptyset$.

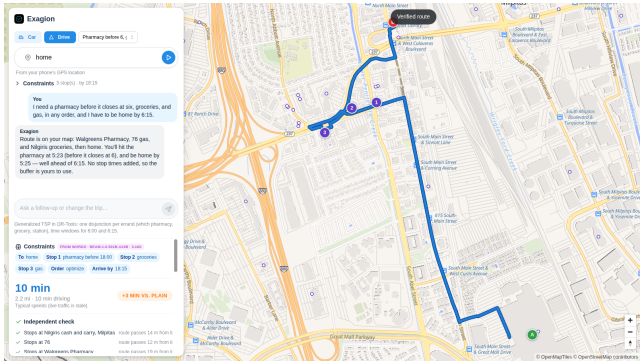
- X_k is increasing; at most 8 rounds
- This run: $k = 0$: 43.5 min, $|B_0| = 122$;
  $k = 1$: 63.1 min, $B_1 = \emptyset$. Total 121 ms.

```
for attempt in range(MAX_EXCLUSION_ROUNDS):
    trip = valhalla.route(points, costing, opts,
                          exclude_locations)
    # /trace_attributes: the edges driven
    edges = valhalla.edges_for_route(trip, costing)
    bad = [e for e in edges
           if any(r.violates(e) for r in rules)]
    if not bad:
        break
    new = [midpoint(e) for e in bad
           if midpoint(e) not in exclude_locations]
    if not new:
        break     # stuck: the verifier reports it
    exclude_locations.extend(new)
```

`nlroute/planner.py` (trimmed)

4. “Three errands, any order, home by 6:15”

- “A pharmacy” has hundreds of answers: choosing one is optimization, not language
- 6 candidates per errand, by detour; one Valhalla matrix over all
- OR-Tools chooses which pharmacy, grocery, and station, and the order
- Result: Walgreens at 5:23 (closes 6:00); home at 5:25 (by 6:15)



Errands as a generalized TSP with time windows

- Groups $G_1, \ldots, G_m$ (pharmacies, groceries, stations); arc times t_{ij}, stop times s_i

$$\begin{aligned} &\min \textstyle\sum_{i,j} t_{ij}\, x_{ij} \\ &\text{s.t. } \textstyle\sum_{i \in G_k} y_i = 1 \quad \forall k \\ &\qquad T_j \geq T_i + s_i + t_{ij} \ \text{ if } x_{ij} = 1 \\ &\qquad T_i \leq \text{close}_i, \quad T_{\text{home}} \leq \text{deadline} \end{aligned}$$

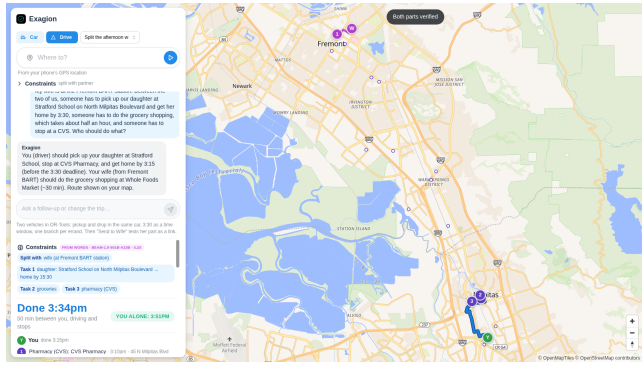
- "Exactly one": a disjunction with max cardinality 1 and penalty 10^7; a skipped group is reported infeasible
- 20 nodes: matrix 0.4 s; guided local search, 1 s limit; path re-costed from the matrix

```
manager = RoutingIndexManager(n, 1, [orig], [home])
routing = RoutingModel(manager)
cb = routing.RegisterTransitCallback(transit)
routing.SetArcCostEvaluatorOfAllVehicles(cb)
routing.AddDimension(cb, 0, 24*3600, True, "Time")
T = routing.GetDimensionOrDie("Time")
for group in groups:
    routing.AddDisjunction(
        [manager.NodeToIndex(i) for i in group],
        10**7, 1)
for node, latest in windows.items():
    T.CumulVar(manager.NodeToIndex(node)) \
     .SetMax(latest)
T.CumulVar(routing.End(0)).SetMax(deadline)
```

`nlroute/solver.py`: `solve_any_order` (trimmed)

5. “Between the two of us, who should do what?”

- Tasks: pick up our daughter at school, home by 3:30; groceries (30 min, as stated); a CVS
- Drivers: me at the Great Mall, my wife at Fremont BART
- Result: me: CVS, school, home 3:15; my wife: Whole Foods
- Both done 3:34; either alone: 3:51 or 4:11



Two vehicles, pickup and delivery

- Vehicles $v \in \{1, 2\}$ start where each driver is; open routes via a virtual end with $t_{ie} = 0$
- Pickup p, drop d: same vehicle, $T_p \leq T_d$, $T_d \leq$ 15:30
- Objective: total time of both drivers plus the span

$$\min \sum_v \sum_{i,j} (t_{ij} + s_i)\, x_{ij}^v + \max_v T_e^v - \min_v T_s^v$$

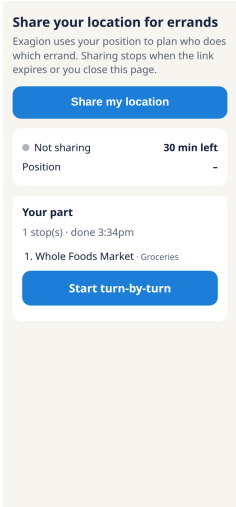
- The span term makes splitting worthwhile. 20 nodes: matrix 0.3 s, search 2 s

```
manager = RoutingIndexManager(n + 1, 2,
                              [me, wife], [end, end])
routing = RoutingModel(manager)
T.SetGlobalSpanCostCoefficient(1)
for group in groups:     # any grocery, any CVS
    routing.AddDisjunction(
        [manager.NodeToIndex(i) for i in group],
        10**7, 1)
p = manager.NodeToIndex(school)
d = manager.NodeToIndex(home)
routing.AddPickupAndDelivery(p, d)
solver.Add(routing.VehicleVar(p)
           == routing.VehicleVar(d))
solver.Add(T.CumulVar(p) <= T.CumulVar(d))
T.CumulVar(d).SetMax(deadline)
```

`nlroute/solver.py`: `solve_team` (trimmed)

Sending a driver their part

- "Send to Wife": a link with an unguessable token, valid 30 minutes; no account, no app
- The page shows her part of the plan and can share her location back
- "Start turn-by-turn": Ferrostar in her browser, on the plan's exact route request (exclusions kept on reroute)
- On expiry the position is deleted and the link returns 410
- Limits: browsers share location only while the page is open; Google Maps location sharing has no API



Every route is checked independently of the planner

- The final line is map-matched again (`/trace_attributes`); checks read those edges
- Checked: stops reached and in order; roads and areas avoided; detour bounds; side of road on arrival; "about *N* min in"; closing times; deadline; pickup before drop in the same car
- A failed check is shown with the measured value; the route is still drawn
- Retry loops fix loud failures
- Silent failures seen in practice: an exclusion polygon rejected by the engine, a stop skipped by the solver, a deadline missed by a relaxed solve

Data: the verifier cannot catch a wrong category

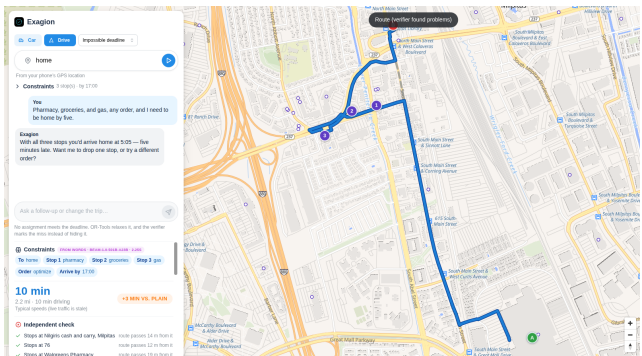
- Overture places with confident, wrong categories:
  - Oportun (≈20 sites): pharmacy; a lender (confidence 0.92–0.99)
  - Evernote: gas station; Biocardia: pharmacy
  - Hills Bros. Coffee Plant: coffee shop; a historic building
  - Gano Excel (Fairfield): coffee shop; a coffee company's business-park unit
- The route does stop where the data says a pharmacy is: geometric checks pass
- Mitigation: a small, visible deny list in code; better: report upstream
- Geocoding ranks names, not meaning: "Oakland airport" → Oakland Airport Executive Hotel, until the OSM tag had to match the named kind

Engines: silent failures

- Valhalla matrices: ≈2% of places null in their entire row and column (8 of 400 shops); all snapped to a parking aisle or service road; `/route` reaches them. Fix: re-snap those points to the street
- Default `max_exclude_polygons_length`: 10 km total perimeter; over it the request is rejected, and the easy fallback silently drops the constraint
- Ollama: tool-call arguments must be objects, not JSON strings; every turn after a tool call failed
- Ollama: two context sizes (8k, 16k) load two copies of a 35B model; on a 12 GB GPU the second load crashed

Impossible requests: say no

- “Home by five” at 4:55 with three errands: OR-Tools relaxes the deadline; the verifier fails it: arrive 5:05
- “A car wash under one minute out of the way”: the closest adds 3.2 min
- Never a plausible route that quietly violates the request



What constraints cannot express yet

- "Downtown": no usable boundary in OSM; a gazetteer circle
- "The Bay Bridge": the name is in `bridge:name`, absent from the routing graph; matched by footprint for a short list
- "Before it closes": no opening hours in the open places data; the driver must state the time
- A named place ("Tartine") is resolved to one point before solving; only categories and chains ("a Safeway") reach the solver as alternatives
- A verifier that shares the planner's definitions shares its mistakes: checks need independent data, not only independent code

Summary

1. Give the language model one job: fill a typed schema. Open engines do the geometry and the optimization.
2. Constraints are predicates over graph edges, not road names; iterate until the route satisfies them.
3. Verify the final route from fresh data, and report failures with the measured value.

Thank you

- Code: github.com/johngcarlsson-usc/TurnByTurn
- Built on Valhalla, OpenStreetMap, Overture Maps, DuckDB, OR-Tools, Photon, Nominatim, Caltrans PeMS, Ferrostar, MapLibre, OpenFreeMap, Open-Meteo, Ollama